

Mastering Next.js & React.js

CẨM NANG TOÀN DIỆN TỪ CƠ BẢN ĐẾN NÂNG CAO

`npx create-next-app@latest`

| Arrow Functions trong ES6

Cú pháp hiện đại

Arrow function giúp rút gọn code và xử lý this một cách tự nhiên hơn.

- Loại bỏ từ khóa function.
- Sử dụng dấu mũi tên =>.
- Implicit return cho các biểu thức đơn giản.

```
// Traditional Function
function hello(name) {
  return "Chào " + name;
}

// Arrow Function
const hello = (name) => `Chào ${name}`;

// React Component
const Header = () => (
  Hello React
);
```

Cấu trúc thư mục Next.js 14

```
my-app/
├── app/                // App Router (New)
│   ├── layout.tsx     // Root Layout
│   ├── page.tsx       // Homepage (/)
│   ├── globals.css
│   └── dashboard/
│       └── page.tsx    // Route: /dashboard
├── public/            // Static files
├── components/        // Reusable UI
├── next.config.js
└── package.json
```

App Router Architecture

Next.js 14 sử dụng cấu trúc thư mục để định nghĩa Routing tự động.

- **app/**: Mọi route nằm trong thư mục này.
- **page.tsx**: File bắt buộc để render UI tại route đó.
- **layout.tsx**: Dùng chung giao diện cho các trang con.

React Components & Props

Truyền dữ liệu qua Props

Props là các tham số được truyền từ component cha xuống component con.

```
// Component Con
const UserCard = ({ name, age }) => {
  return (
    <div>
      Tên: {name}
      Tuổi: {age}
    </div>
  );
};

// Component Cha
const App = () => {
  return (
    <UserCard
      Tên: {name}
      Tuổi: {age}
    />
  );
};

export default UserCard;
```

Quản lý State với useState

```
import { useState } from 'react';

function Counter() {
  // Khai báo state count
  const [count, setCount] = useState(0);

  return (
    <div>
      Bạn bấm {count} lần
      <button onClick={() => setCount(count + 1)}>
        Tăng
      </button>
    </div>
  );
}
```

Bộ nhớ của Component

State cho phép component ghi nhớ thông tin và tự động render lại khi dữ liệu thay đổi.

- **useState**: Trả về 1 mảng gồm [giá trị, hàm cập nhật].
- **Rerender**: Gọi hàm cập nhật state sẽ kích hoạt UI update.

Xử lý Side Effects (useEffect)

Vòng đời & Side Effects

Dùng cho các tác vụ như: Fetch API, Đăng ký sự kiện, hay thay đổi DOM thủ công.

- `[]`: Chạy 1 lần duy nhất khi mount.
- `[var]`: Chạy lại khi var thay đổi.

```
useEffect(() => {  
  // Gọi API khi trang load  
  fetchData().then(data => setData(data));  
  
  // Cleanup function (optional)  
  return () => cleanup();  
}, []); // Dependency array
```

Server Components & Data Fetching

```
// app/products/page.tsx (Server Component)
async function ProductsPage() {
  // Fetch trực tiếp trên Server
  const res = await fetch('https://api.../products');
  const products = await res.json();

  return (
    {products.map(p => {p.name})}
  );
}
```

Ưu việt của Next.js

Server Components cho phép gọi API trực tiếp trong function async mà không cần useEffect.

- **Tốc độ:** Giảm lượng JavaScript gửi xuống Client.
- **SEO:** HTML được render sẵn từ server.

Redux Toolkit: createSlice

Quản lý State toàn cục

Redux giúp chia sẻ dữ liệu giữa các component không có quan hệ cha con.

- **Slice:** Gộp Action và Reducer vào 1 nơi.
- **Immer:** Cho phép "mutate" state ảo cực kỳ tiện lợi.

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: state => { state.value += 1 }
  }
});

export const { increment } = counterSlice.actions;
export default counterSlice.reducer;
```


Redux Toolkit: Store & Provider

```
// store.js
export const store = configureStore({
  reducer: { counter: counterReducer }
});

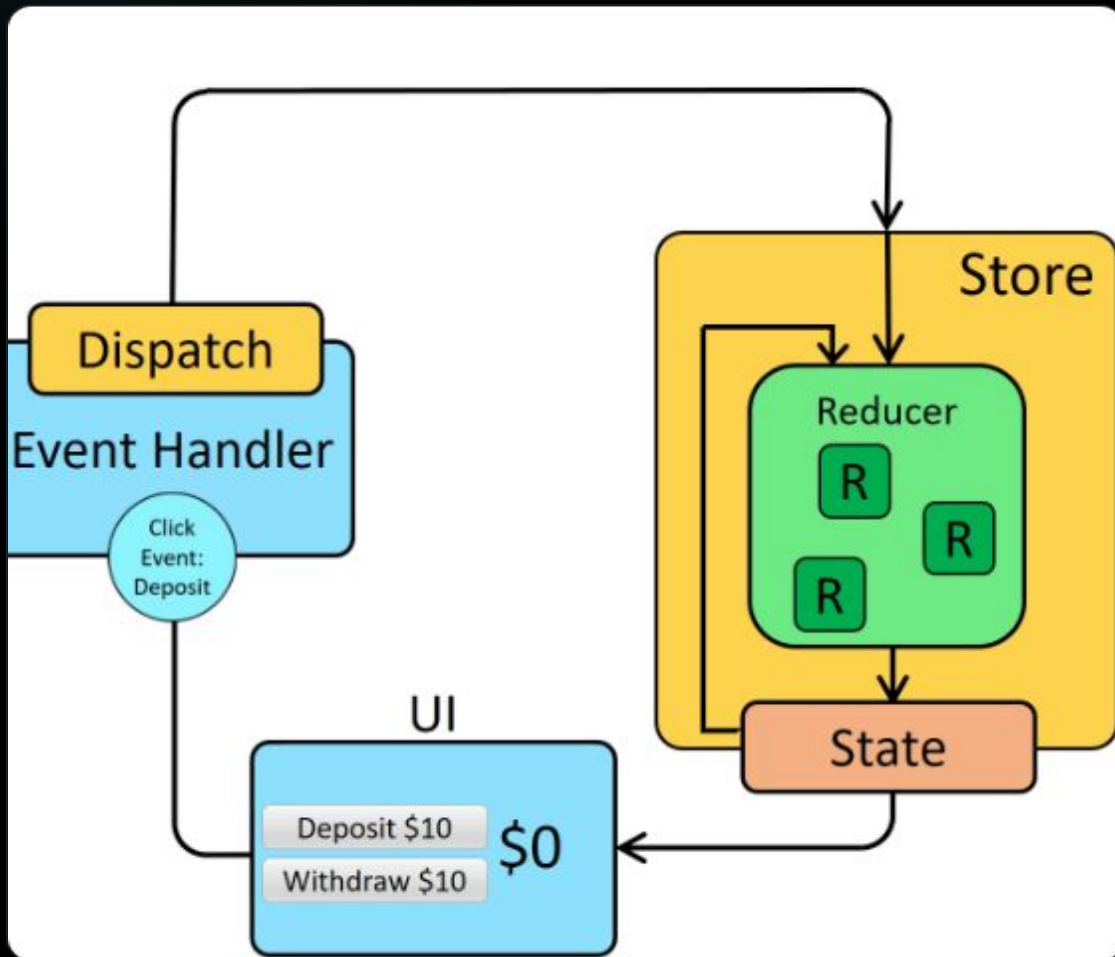
// index.js / layout.js
```

Tích hợp vào Ứng dụng

Tạo Store và bọc ứng dụng trong Provider để mọi component có thể truy cập dữ liệu.

Sử dụng useSelector để lấy data và useDispatch để gửi hành động.

Redux Flow & Next.js Diagram



Luồng dữ liệu một chiều

Dữ liệu trong Redux luôn tuân thủ luồng nghiêm ngặt giúp dễ dàng debug.

- **Action:** Sự kiện xảy ra.
- **Reducer:** Tính toán state mới.
- **Store:** Cập nhật trạng thái và thông báo UI.

Client vs Server Components

```
'use client'; // Phải có dòng này!  
import { useState } from 'react';  
  
export default function InteractiveBtn() {  
  return Click;  
}
```

Server (Default): Tốt cho SEO, Performance, Data fetching.

Client: Bắt buộc khi dùng Hook, Event Listeners (onClick) hoặc Browser API.

